



Personal Notification Agent

Agente inteligente de seguimiento y alertas automáticas

github.com/Rubifen/personal-notification-agent/

| ¿Qué es este Agente?

Automatización con Lenguaje Natural

Resuelve el seguimiento manual de datos (clima, web, etc.)
permitiendo alertas configuradas mediante **voz o texto**.

Se ejecuta en segundo plano en Python, comprendiendo la
intención del usuario para decidir cuándo y cómo avisar.

| Capacidades Principales



Comprensión

Extrae parámetros y selecciona herramientas automáticamente analizando comandos de formato libre.



Dictado por Voz

Integración de Speech-to-Text para dictar reglas de notificación sin necesidad de escribir.



Multicanal

Notificaciones vía Telegram, WhatsApp (CallMeBot), Email y alertas nativas de escritorio.

| Stack Tecnológico

Categoría	Tecnología / Herramienta
Lenguaje & IA	Python 3, API OpenRouter (LLM), Google Antigravity
Interfaz	CustomTkinter (GUI Moderna)
Audio & Datos	SpeechRecognition, SQLite3, BeautifulSoup4 (Scraping)
Notificaciones	Plyer, HTTP Requests (Telegram/WhatsApp), SMTP

| El "Cerebro" del Agente

```
if language is None:
    raise Exception("Input language could not be determined")
    return None
input = self.parseInputToLanguageModel(inputString, inputLanguage)
parsedInput = input or not self.model:
    return None
context.append(parsedInput) # Add new conversation entry to context
return (self.model.generateLLMOutput(parsedInput), context)

def parseInputToLanguageModel(inputString, inputLanguage, context):
    self.model is None or self.model.language != inputLanguage:
        # LLM is not initialised or has wrong language, load LLM
        self.model = self.loadAILanguageModelFromDatabase(inputLanguage)
        if self.model is None or not self.runModelSelfDiagnosticTest():
            raise Exception("AI language model load failed")
            return None
        self.model.setLLMContext(context) # Put past conversation context
        self.llmInputParser = self.model.getInputParser()
        return self.llmInputParser.parseInput(inputString)

def generateLLMOutput(parsedInput):
    llmContext = self.model.getLLMContext()
    llmResponse = self.model.convertInputToIntermediateRepresentation(parsedInput, llmContext)
    if llmResponse is None:
```

Modelos LLM & Razonamiento

Utiliza la API de **OpenRouter** para analizar comandos como:

"Avísame si llueve en Manacor dentro de una hora".

Procesamiento:

1. Análisis de intención.
2. Extracción de parámetros (Lugar, Tiempo).
3. Selección de herramienta (API Clima).
4. Programación de frecuencia.

| Arquitectura Modular

core/: El núcleo. Agent (IA), Task Manager (SQLite) y Background Scheduler.

gui/: Interfaz CustomTkinter con hilos seguros para evitar bloqueos.

tools/: Scripts especializados como 'clima_api'.

main.py: Punto de entrada y orquestador principal.

| Flujo de Trabajo IA



Uso de **Antigravity de Google** para automatizar gran parte del proceso.

| Retos y Soluciones



Tiempo Relativo

Solución: Inyección de contexto de hora local del sistema como mensaje oculto para que la IA entienda "dentro de una hora".



Hilos y GUI

Solución: Uso de callbacks y métodos seguros para evitar que hilos secundarios (grabación/scheduler) bloqueen la interfaz.

| Reflexión sobre el uso de IA

| La IA ha sido clave para especificar ideas genéricas. Aunque el proceso es casi automático con Antigravity, la especificidad es vital para evitar interpretaciones erróneas.

| Limitaciones y Futuro

Hacia Agentes Interactivos

- ✓ **Acción Real:** Capacidad de escritura y modificación, no solo lectura.
 - ✓ **Sub-agentes:** Desarrollo de bots de respuesta automática.
 - ✓ **Mejora Web:** Superar limitaciones de selectores CSS manuales y captchas.
-

| Instalación en 3 Pasos

- ✓ Clonar el repositorio y entrar en la carpeta raíz.
- ✓ Instalar requisitos: `pip install -r requirements.txt`
- ✓ Configurar el archivo `.env` con tus claves de OpenRouter y notificaciones.
- ✓ Ejecutar: `python main.py`

